

MDT915 — Individual Contribution Report

Student Name: Seif Sid Ali Maloufi

Student ID: 8718179

Project: CharityFlow — Transparent Charity Donation Platform

Course: MDT915 Blockchain Practitioner

1. My Specific Contributions

Smart Contract Development

- Designed and implemented the CharityDonation.sol smart contract in Solidity 0.8.20.
- Defined core data structures: Campaign, Donation, ProofOfWork, WithdrawalRecord, ImpactReport.
- Implemented full campaign lifecycle: creation, donation (with 2% platform fee), withdrawal with mandatory proof hash, proof submission, and impact reporting.
- Added platform fee mechanism (200 basis points) and admin functions for charity, campaign, and proof verification.
- Implemented refund logic for cancelled campaigns and deadline enforcement.
- Used events for frontend integration and activity feeds.

Frontend & Web3 Integration

- Built the React application using Vite, React Router, and Ethers.js v6.
- Implemented all pages: Landing, Campaign List, Campaign Detail, Donor Dashboard, Charity Portal.

- Created `contract.js` with multi-network support (Hardhat local + Sepolia).
- Integrated Web3Context for MetaMask connection and contract instance creation.
- Implemented real on-chain activity feeds in Donor and Charity dashboards.
- Added network switch UI in Navbar when user is on the wrong network.

IPFS Integration

- Integrated Pinata API in `ipfs.js` (`uploadToIPFS`, `ipfsUrl`).
- Created IPFSUpload component for single and multiple file uploads.
- Wired IPFS into campaign cover images, proof documents, withdrawal receipts, and impact report media.

Testing & Quality Assurance

- Wrote 41 unit tests using Mocha, Chai, and Hardhat Network Helpers.
- Covered deployment, campaign creation, donations, proofs, withdrawals, refunds, impact reports, admin functions, and deadline enforcement.

Deployment & Configuration

- Configured Hardhat for local development and Sepolia deployment.
- Deployed contract to Sepolia testnet.
- Set up deploy script to auto-update frontend `.env` with contract address.

2. Time Investment

Activity	Estimated Hours
Smart contract design & implementation	12
Unit testing	6
Frontend structure & pages	10
Web3/Ethers.js integration	6
IPFS/Pinata integration	4
Hardhat configuration & deployment	3
Documentation & README	4
Debugging & refinement	5
Total	50

3. Challenges & Solutions

Challenge	Solution
Ensuring withdrawals require proof of spending	Implemented proofHash parameter in withdrawFunds() and linked it to WithdrawalRecord so every withdrawal is traceable on-chain.
Contract address per network	Created CONTRACT_BY_CHAIN map in contract.js and passed chainId from Web3Context to getContract().
Empty activity feed on Sepolia	Updated .env with Sepolia deployment address and ensured deploy script auto-updates it.
Manual IPFS hash entry	Integrated Pinata API and built IPFSUpload component so users upload files directly.
Node version compatibility	Documented in README and used Node 20 for deployment where possible.

4. Learning Outcomes

- Gained hands-on experience with Solidity smart contract development.
- Understood gas considerations and event emission for frontend integration.
- Learned Hardhat workflow: compile, test, deploy to local and Sepolia testnet.
- Applied blockchain concepts: escrow, verification, immutable ledger.

- Practiced Ethers.js v6 for connecting React to Ethereum.
- Integrated IPFS via Pinata for decentralized storage.

5. Team Collaboration

- Jyldyz handled report writing, documentation, and academic theory; I focused on implementation.
- Shared contract ABI and event names so the report could accurately describe the system.
- Coordinated on project structure and README content for clarity.
- Reviewed each other's work before submission.

Date: 14/03/2026